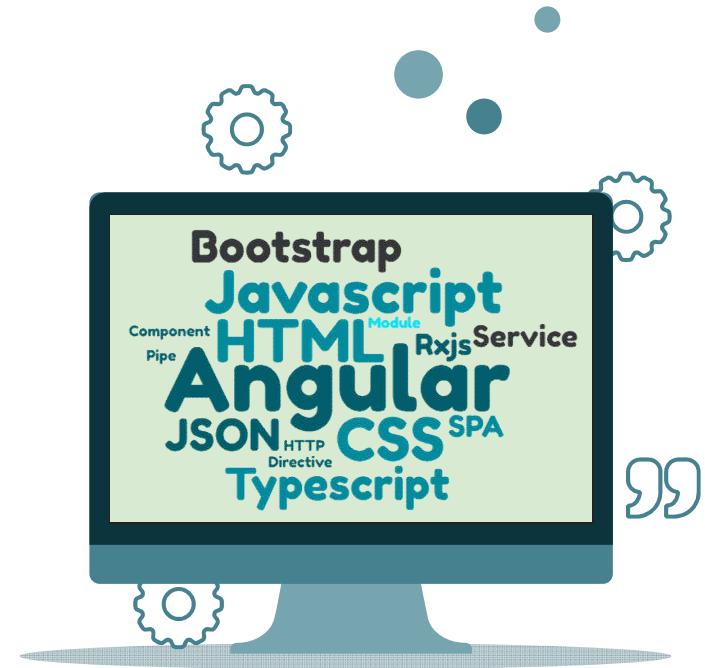


05

Routage



Chapitre 5 Routage

01

Routing & Navigation

02

Guards

03

Atelier 3

01

Routing & Navigation



Routage (Routing)

- Afin de permettre aux utilisateurs de garder leurs habitudes de navigation en visitant une SPA (Single Page Application), il est nécessaire d'utiliser un système de Routage.
- Grâce à ce système, les utilisateurs peuvent :
 - utiliser l'historique de leur navigateur (par exemple les boutons **back** et **next**),
 - partager des liens,
 - ajouter une vue à leurs favoris,
 - ouvrir une vue dans une nouvelle fenêtre via le menu contextuel,
 - ...

Angular Routing

Principe

- Angular fournit nativement un module de Routage
- Le routeur d'Angular permet la navigation d'une vue à l'autre lorsque les utilisateurs exécutent des tâches d'application.
- C'est un service facultatif qui présente une vue de composant pour une URL donnée
- Il ne fait pas partie du noyau → besoin de la bibliothèque, [@angular/router](#).

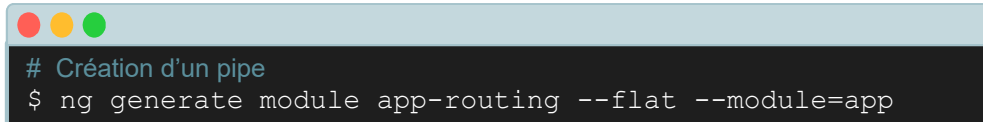
```
E:\AngularProjects>ng new first-project
? Would you like to add Angular routing? Yes
? Which stylesheet format would you like to use? CSS
```

```
TS app-routing.module.ts ×
src > app > TS app-routing.module.ts > ...
1 import { NgModule } from '@angular/core';
2 import { Routes, RouterModule } from '@angular/router';
3
4
5 const routes: Routes = [];
6
7 @NgModule({
8   imports: [RouterModule.forRoot(routes)],
9   exports: [RouterModule]
10 })
11 export class AppRoutingModule { }
```

```
TS app.module.ts ×
src > app > TS app.module.ts > AppModule
6 import { AppRoutingModule } from './app-routing.module';
7 import { AppComponent } from './app.component';
8 import { AboutComponent } from './about/about.component';
9 import { ContactsComponent } from './contacts/contacts.component';
10 import { AboutService } from './services/about.service';
11
12 @NgModule({
13   declarations: [
14     AppComponent,
15     AboutComponent,
16     ContactsComponent
17   ],
18   imports: [
19     BrowserModule,
20     FormsModule,
21     AppRoutingModule
22   ],
```

Angular Routing

- Et si on n'a pas accepté qu'il soit généré à la création du projet ?
 - Pour créer un module de routage

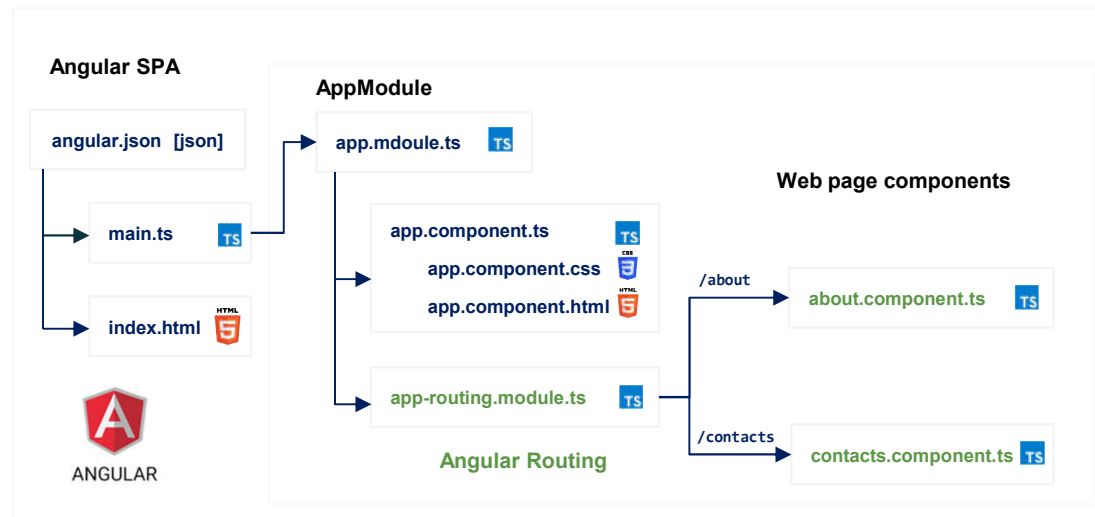


```
# Création d'un pipe
$ ng generate module app-routing --flat --module=app
```

- **--flat** : pour placer le fichier dans src/app et éviter de créer un répertoire propre à ce module.
- **--module=app** : pour enregistrer ce module dans le tableau `imports` de `AppModule`.

Schéma de routage de notre application

- Un schéma pour récapituler ce que nous allons faire.



Etant donnée cette configuration, quand l'utilisateur tape <http://localhost:4200/about> :

- Le routeur cherche et charge le composant `AboutComponent` et l'affiche dans un élément `<router-outlet> ... </router-outlet>`
- Cet élément est sensé se trouver dans la vue du `AppComponent`.

Mise en place du Routing : 1/ Tag Base

- Avant toute chose, il est nécessaire d'ajouter le tag base au head du fichier `index.html` de l'application :

```
<> index.html ×
src > <> index.html > html > body
1  <!doctype html>
2  <html lang="en">
3  <head>
4    <meta charset="utf-8">
5    <title>FirstProject</title>
6
7    <!-- balise permettant le routage -->
8    <base href="/">
9    <meta name="viewport" content="width=device-width, initial-scale=1">
10   <link rel="icon" type="image/x-icon" href="favicon.ico">
11 </head>
12 <body>
13
```

- Ce tag indique la base du path à partir de laquelle le **Routage d'Angular** entre en jeu.

Mise en place du Routing : 2/ Configuration

2.1 Configuration du Routage

- La configuration du routage est déléguée au module RouterModule (app-routing.module.ts) lors de son importation par le AppModule

```
TS app-routing.module.ts X
src > app > TS app-routing.module.ts > ...
1  import { NgModule } from '@angular/core';
2  import { Routes, RouterModule } from '@angular/router';
3
4  const routes: Routes = [];
5
6  @NgModule({
7    imports: [RouterModule.forRoot(routes)],
8    exports: [RouterModule]
9  })
10 export class AppRoutingModule { }
```

- RouterModule.forRoot() : est utilisé pour initialiser le système de routage de l'application Angular en fournissant les définitions des routes et en configurant le module de routage principal.

Mise en place du Routing : 2/ Configuration

2.2 Configuration des routes

- La configuration des **routes** est une liste d'objets avec les propriétés suivantes :
 - Component** : Le composant à charger (Obligatoire)
 - path** : URL (Obligatoire)
 - redirectTo** : Redirection vers une autre route.

Info

`pathMatch: 'full'` : cette configuration assure qu'une fois que l'utilisateur accède à la page principale du site `http://localhost:4200/`, il est redirigé automatiquement vers la route `/about`, mais seulement si l'URL est vide dans son intégralité.

©MAD

```
TS app-routing.module.ts X
src > app > TS app-routing.module.ts > ...
1 import { NgModule } from '@angular/core';
2 import { Routes, RouterModule } from '@angular/router';
3 import { AboutComponent } from '../about/about.component';
4 import { ContactsComponent } from '../contacts/contacts.component';
5
6 const routes: Routes = [
7   { path: 'about', component: AboutComponent },
8   { path: 'contacts', component: ContactsComponent },
9   { path: '', redirectTo: '/about', pathMatch: 'full' }
10 ];
11
12 @NgModule({
13   imports: [RouterModule.forRoot(routes)],
14   exports: [RouterModule]
15 })
16 export class AppRoutingModule {
17 }
```

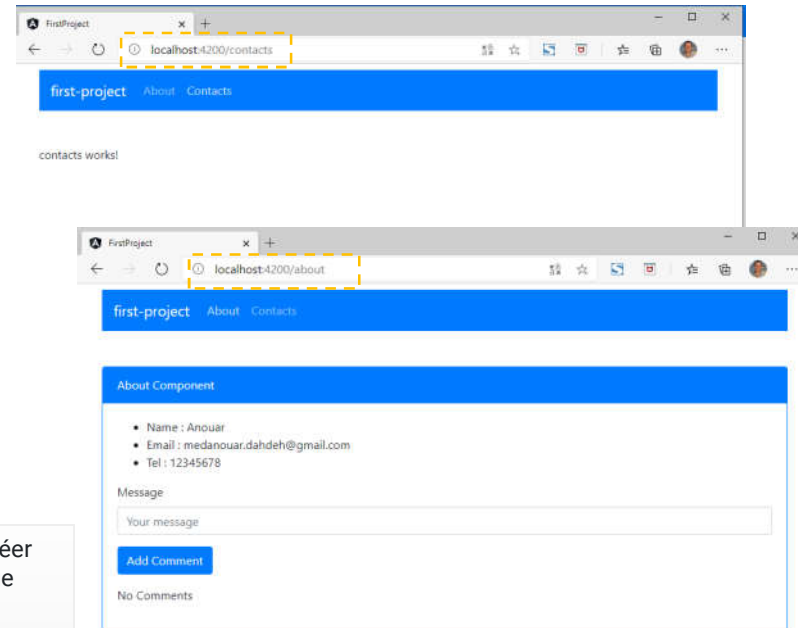
Mise en place du Routing : 2/ Configuration

2.3 Création de liens

```
app.component.html X
src > app > app.component.html > ...
1
2 <div class="container mt-2">
3   <nav class="navbar navbar-expand-lg navbar-dark bg-primary">
4     <a class="navbar-brand" href="#">{{title}}</a>
5     <button class="navbar-toggler" type="button" data-toggle="collapse" data-target="#navbarNav"
6       aria-controls="navbarNav" aria-expanded="false" aria-label="Toggle navigation">
7       <span class="navbar-toggler-icon"></span>
8     </button>
9     <div class="collapse navbar-collapse" id="navbarNav">
10      <ul class="navbar-nav">
11        <li class="nav-item"><a class="nav-link" routerLink="/about">About</a></li>
12        <li class="nav-item"><a class="nav-link" routerLink="/contacts">Contacts</a></li>
13      </ul>
14    </div>
15  </nav>
16 </div>
17 <div class="container mt-5">
18   <router-outlet></router-outlet>
19 </div>
```

Cette directive indique où afficher les pages (vues des composants)

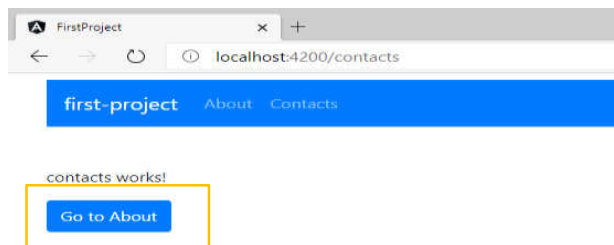
La directive **routerLink** permet de créer le lien vers les pages souhaitées (les vue des composants) .



Mise en place du Routing : Navigation

- Il est également possible de naviguer entre les différentes routes en utilisant la méthode `navigate()` du service `Router`.
- Le service `Router` doit être injecté dans la classe composant.

```
contacts.component.html ×
src > app > contacts > contacts.component
1 <p>contacts works!</p>
2
3 <button class="btn btn-primary"
4       (click) = "onAbout()">
5   Go to About
6 </button>
```

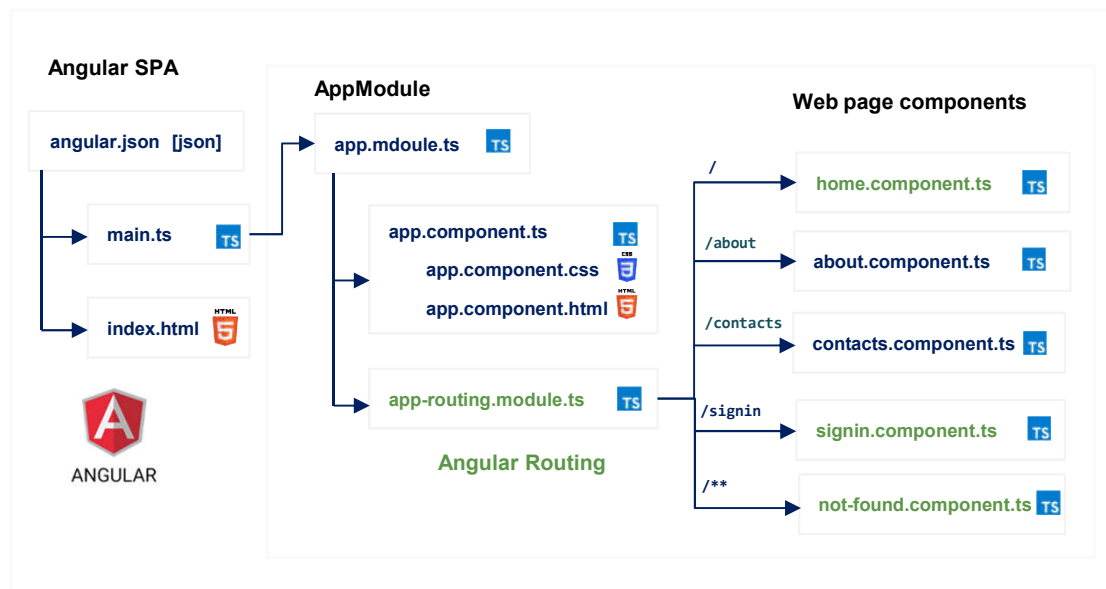


```
TS contacts.component.ts ×
src > app > contacts > TS contacts.component.ts > ...
1 import { Component, OnInit } from '@angular/core';
2 import { Router } from '@angular/router';
3
4 @Component({
5   selector: 'app-contacts',
6   templateUrl: './contacts.component.html',
7   styleUrls: ['./contacts.component.css']
8 })
9 export class ContactsComponent implements OnInit {
10
11   constructor(private router : Router) { }
12
13   ngOnInit() {
14   }
15
16   onAbout(){
17     this.router.navigate(['/about']);
18   }
19 }
```

Notre FirstProject: Plus de composants

- Nous allons créer 3 autres composants qui seront utilisés dans le routing :
 - Home, Signin et NotFound

```
# Composant Home
$ ng generate component home
# Composant Singin
$ ng generate component signin
# Composant NotFound
$ ng generate component notFound
```



Notre FirstProject: Plus de composants

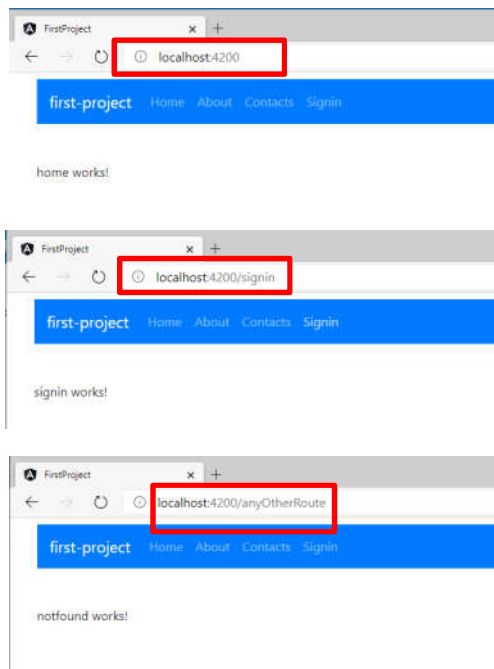
- Mise à jour du app-routing.module.ts

```
TS app-routing.module.ts X
src > app > TS app-routing.module.ts > ...
1  import { NgModule } from '@angular/core';
2  import { Routes, RouterModule } from '@angular/router';
3  import { AboutComponent } from './about/about.component';
4  import { ContactsComponent } from './contacts/contacts.component';
5  import { HomeComponent } from './home/home.component';
6  import { NotFoundComponent } from './notfound/notfound.component';
7  import { SigninComponent } from './signin/signin.component';
8
9  const routes: Routes = [
10   { path: '', component: HomeComponent },
11   { path: 'contacts', component: ContactsComponent },
12   { path: 'about', component: AboutComponent },
13   { path: 'signin', component: SigninComponent },
14   { path: '**', component: NotFoundComponent }
15 ];
16
17 @NgModule({
18   imports: [RouterModule.forRoot(routes)],
19   exports: [RouterModule]
20 })
21 export class AppRoutingModule {
22 }
```

Page par défaut (<http://localhost:4200>)

En cas d'erreur d'url : rediriger l'utilisateur vers une page d'erreur

Notre FirstProject: Plus de composants



```
app.component.html X
src > app > app.component.html > ...
1
2 <div class="container mt-2">
3   <nav class="navbar navbar-expand-lg navbar-dark bg-primary">
4     <a class="navbar-brand" href="#">{{title}}</a>
5     <button class="navbar-toggler" type="button" data-toggle="collapse" data-target="#navbarNav"
6       aria-controls="navbarNav" aria-expanded="false" aria-label="Toggle navigation">
7       <span class="navbar-toggler-icon"></span>
8     </button>
9     <div class="collapse navbar-collapse" id="navbarNav">
10      <ul class="navbar-nav">
11        <li class="nav-item"><a class="nav-link" routerLink="/">Home</a></li>
12        <li class="nav-item"><a class="nav-link" routerLink="/about">About</a></li>
13        <li class="nav-item"><a class="nav-link" routerLink="/contacts">Contacts</a></li>
14        <li class="nav-item"><a class="nav-link" routerLink="/signin">Signin</a></li>
15      </ul>
16    </div>
17  </nav>
18 </div>
19 <div class="container mt-5">
20   <router-outlet></router-outlet>
21 </div>
```

Routing : Paramètres de requête

- Deux formes de paramètres de requête
 - `/chemin/param1/param2` (map Request)
 - `/chemin?var1=value1&var2=value2` (query request)
- Pour ces deux formes de paramètres
 - Deux manières différentes de définir les routes
 - Deux objets différents permettant de récupérer les valeurs respectives
 - `paramMap` pour `/chemin/param1/param2`
 - `queryParamMap` pour `/chemin?var1=value1&var2=value2`

Routing : paramètres de requête (map Request)

- Définissons une route de la forme `/chemin/param1/param2` dans `app-routing.module.ts`

```
const routes: Routes = [  
  {path: 'about', component: AboutComponent},  
  {path: 'contacts', component: ContactsComponent},  
  {path: 'contacts/:param1/:param2', component: ContactsComponent},  
  {  
    path: '',  
    redirectTo: '/about',  
    pathMatch: 'full'  
  }  
];
```

- Pour récupérer les paramètres d'une route de la forme `contacts/:param1/:param2`, il faut :
 - aller dans le composant concerné (ici, `contacts.component.ts`)
 - Injecter par constructeur la classe `ActivatedRoute`
 - utiliser un objet de cette classe dans la méthode `ngOnInit()`
 - soit par l'intermédiaire d'un objet `paramMap` pour récupérer les paramètres (solution avec les observables (asynchrone))
 - soit par l'intermédiaire d'un objet `params` pour récupérer les paramètres (*solution avec les snapshot (instantanée)*)

Routing : paramètres de requête (map Request)

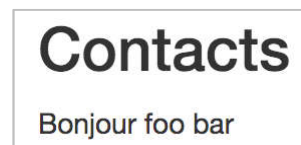
- par l'intermédiaire d'un objet `paramMap` pour récupérer les paramètres
(*solution avec les observables (asynchrone)*)

```
app > contacts > TS contacts.component.ts > ...
import { Component, OnInit } from '@angular/core';
import { ActivatedRoute } from '@angular/router';

@Component({
  selector: 'app-contacts',
  templateUrl: './contacts.component.html',
  styleUrls: ['./contacts.component.css']
})
export class ContactsComponent implements OnInit {
  param1: any;
  param2: any;
  constructor(private route: ActivatedRoute) { }
  ngOnInit() {
    this.route.paramMap.subscribe(res => {
      this.param1 = res.get('param1');
      this.param2 = res.get('param2');
      console.log(this.param1 + ' ' + this.param2);
    });
  }
}
```



```
app > contacts > <> contacts.component.html > ...
<p> Bonjour {{ param1 }} {{ param2 }} </p>
```



Routing : paramètres de requête (map Request)

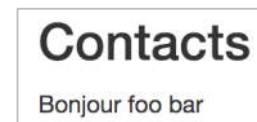
- par l'intermédiaire d'un objet `params` pour récupérer les paramètres (*solution avec les snapshot (instantanée)*)

```
import { Component, OnInit } from '@angular/core';
import { ActivatedRoute } from '@angular/router';

@Component({
  selector: 'app-contacts',
  templateUrl: './contacts.component.html',
  styleUrls: ['./contacts.component.css']
})
export class ContactsComponent implements OnInit {
  param1: any;
  param2: any;
  constructor(private route: ActivatedRoute) { }
  ngOnInit() {
    this.param1 = this.route.snapshot.params.param1;
    this.param2 = this.route.snapshot.params.param2;
    console.log(this.param1 + ' ' + this.param2);
  }
}
```



```
app > contacts > <> contacts.component.html > ...
<p> Bonjour {{ param1 }} {{ param2 }} </p>
```



Routing : paramètres de requête (param Request)

- Pour récupérer les paramètres d'une route de la forme `contacts?param1=value1¶m2=value2`,
- Il faut :
 - Aller dans le composant concerné (ici, `contacts.component.ts`)
 - Injecter par constructeur la classe `ActivatedRoute`
 - Utiliser un objet de cette classe dans la méthode `ngOnInit()`
 - soit par l'intermédiaire d'un objet `queryParamMap` pour récupérer les paramètres (solution avec les observables)
 - soit par l'intermédiaire d'un objet `queryParams` pour récupérer les paramètres (solution avec les snapshot)

Routing : paramètres de requête (param Request)

- Soit par l'intermédiaire d'un objet `queryParams` pour récupérer les paramètres (solution avec les observables)

```
import { Component, OnInit } from '@angular/core';
import { ActivatedRoute } from '@angular/router';

@Component({
  selector: 'app-contacts',
  templateUrl: './contacts.component.html',
  styleUrls: ['./contacts.component.css']
})
export class ContactsComponent implements OnInit {
  param1: any;
  param2: any;
  constructor(private route: ActivatedRoute) {}
  ngOnInit() {
    this.route.queryParams.subscribe(res => {
      this.param1 = res.get('param1');
      this.param2 = res.get('param2');
      console.log(this.param1 + ' ' + this.param2);
    });
  }
}
```



```
app > contacts > <> contacts.component.html > ...
<p> Bonjour {{ param1 }} {{ param2 }} </p>
```



Routing : paramètres de requête (param Request)

- Soit par l'intermédiaire d'un objet `queryParams` pour récupérer les paramètres (solution avec les snapshot)

```
import { Component, OnInit } from '@angular/core';
import { ActivatedRoute } from '@angular/router';

@Component({
  selector: 'app-contacts',
  templateUrl: './contacts.component.html',
  styleUrls: ['./contacts.component.css']
})
export class ContactsComponent implements OnInit {
  param1: any;
  param2: any;
  constructor(private route: ActivatedRoute) { }
  ngOnInit() {
    this.param1 = this.route.snapshot.queryParams.param1;
    this.param2 = this.route.snapshot.queryParams.param2;
    console.log(this.param1 + ' ' + this.param2);
  }
}
```



```
app > contacts > < contacts.component.html > ...
<p> Bonjour {{ param1 }} {{ param2 }} </p>
```

Contacts
Bonjour foo bar

Routing : Création de lien avec paramètres

- Dans le TypeScript

```
this.lienContact = "/contacts/" + this.param1 + "/" + this.param2;
```

- Référence à un lien avec **l'interpolation**

```
<a routerLink='{ { lienContact } }'> Contacts </a>
```

- Référence à un lien **Property Binding**

```
<a [routerLink]='lienContact'> Contacts </a >
```

- Ou

```
<a [routerLink]="['/contacts',param1,param2]"> Contacts </a>
```

- **Pour construire des chemins de la forme /chemin?param1=value1¶m2=value2:**

```
<a [routerLink]="['/contacts']" [queryParams]="{param1:'Tounsi',param2:'Adam'}">
```

Exercice : Contacts & ContactDetail Component

#04

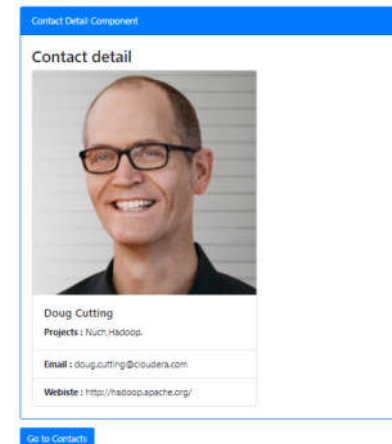
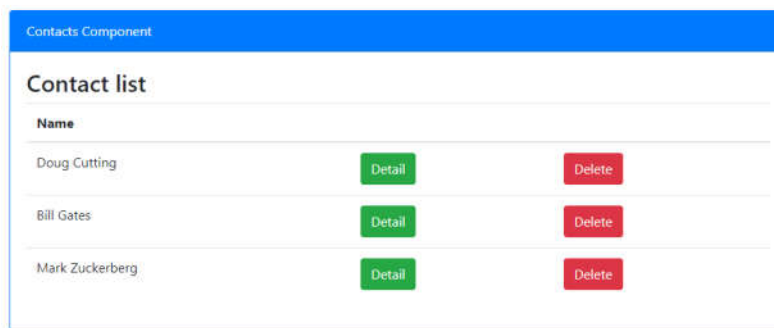
Entant que utilisateur je veux
voir la liste des contacts

#05

Entant que utilisateur je veux
voir les détails d'un contact

#06

Entant que utilisateur je veux
supprimer un contact



Solution

#04

Entant que utilisateur je veux
voir la liste des contacts

#05

Entant que utilisateur je veux
voir les détails d'un contact

#06

Entant que utilisateur je veux
supprimer un contact

1

Créer deux fichiers **contact.ts**
et **contacts.ts** sous le dossier **/shared**

```
TS contact.ts X
src > app > Shared > TS contacts > ...
1 export class Contact {
2   id: number;
3   name: string;
4   email: string;
5   website: string;
6   projects: string[];
7   image: string;
8   featured: boolean;
9 }
```

©MAD

2

Création d'un service pour la gestion des contacts (CRUD)
\$ ng generate service services/contact

```
TS contacts.ts X
src > app > shared > TS contacts > ...
1 import { Contact } from './contact';
2 export const CONTACTS: Contact[] = [
3
4   {
5     id: 1,
6     name: "Doug Cutting",
7     email: "doug.cutting@cloudera.com",
8     website: "http://hadoop.apache.org/",
9     projects: ["Nuch", "Hadoop"],
10    image: "./assets/images/doug_cutting.jpg",
11    featured: true
12  },
13  {
14    id: 2,
15    name: "Bill Gates",
16    email: "bill.gates@outlook.com",
17    website: "https://www.gatesnotes.com/",
18    projects: ["Microsoft", "Windows"],
19    image: "./assets/images/bill_gates.jpg",
20    featured: true
21  },
22  {
23    id: 3,
24    name: "Mark Zuckerberg",
25    email: "bill.gates@facebook.com",
26    website: "https://www.mark-zuckerberg.com/",
27    projects: ["Facebook", "Instagram"],
28    image: "./assets/images/mark_zuckerberg.jpg",
29    featured: true
30  }
31 ]
```

```
TS contact.service.ts U X
src > app > services > TS contactservices > ...
1 import { Injectable } from '@angular/core';
2 import { Contact } from '../shared/contact';
3 import { CONTACTS } from '../shared/contacts';
4
5 @Injectable({
6   providedIn: 'root'
7 })
8 export class ContactService {
9   contacts: Contact[] = CONTACTS
10
11   constructor() { }
12
13   getContacts(): Contact[] { } #04
14   return this.contacts;
15 }
16
17 getContactById(id: number): Contact {
18   return this.contacts.find(contact => contact.id == id); #05
19 }
20
21 deleteContactById(id: number): void {
22   let index = this.contacts.findIndex(contact => contact.id == id) #06
23   this.contacts.splice(index, 1);
24 }
25 }
```

Solution

#04

Entant que utilisateur je veux
voir la liste des contacts

#05

Entant que utilisateur je veux
voir les détails d'un contact

#06

Entant que utilisateur je veux
supprimer un contact

3 Mettez à jour ContactsComponent

```
TS contacts.component.ts X
src > app > contacts > TS contacts.component.ts > ...
1 import { Component, OnInit } from '@angular/core';
2 import { Router } from '@angular/router';
3 import { ContactService } from '../services/contact.service';
4 import { Contact } from '../Shared/contact';
5
6 @Component({
7   selector: 'app-contacts',
8   templateUrl: './contacts.component.html',
9   styleUrls: ['./contacts.component.css']
10 })
11 export class ContactsComponent implements OnInit {
12   contacts : Contact[];
13   constructor( private router: Router, private contactService: ContactService) { }
14   ngOnInit(): void {
15     this.contacts=this.contactService.getContacts(); #04
16   }
17   onDelete(id:number){
18     this.contactService.deleteContactById(id); #06
19   }
20   onAbout() {
21     this.router.navigate(['/about']);
22   }
23 }
```

```
contacts.component.html X
src > app > contacts > contacts.component.html > ...
1 <div class="card border-primary mb-3">
2   <div class="card-header bg-primary text-white "> Contacts Component</div>
3   <div class="card-body">
4     <div *ngIf="contacts.length>0 ; else noContacts">
5       <h3>Contact list</h3>
6       <table class="table table-hover">
7         <thead>
8           <tr>
9             <th scope="col">Name</th>
10            <th scope="col"></th>
11            <th scope="col"></th>
12          </tr>
13        </thead>
14        <tbody>
15          <tr *ngFor="let c of contacts">
16            <td>{{c.name}}</td>
17            <td><a class="btn btn-success" routerLink="/contacts/{{c.id}}">Detail</a></td> #05
18            <td><a class="btn btn-danger" (click)="onDelete(c.id)">Delete</a></td>
19          </tr>
20        </tbody>
21      </table>
22    </div>
23    <ng-template #noContacts>
24      <p>No Contacts</p>
25    </ng-template>
26  </div>
27 </div>
28 <button class="btn btn-primary" (click)="onAbout()">
29   Go to About
30 </button>
```

Solution

#04

Entant que utilisateur je veux
voir la liste des contacts

#05

Entant que utilisateur je veux
voir les détails d'un contact

#06

Entant que utilisateur je veux
supprimer un contact

4 Créez ContactDetailComponent

```
TS contact-detail.component.ts X
src > app > contact-detail > TS contact-detail.component.ts > ContactDetailComponent > onContacts
1 import { Component, OnInit } from '@angular/core';
2 import { ActivatedRoute, Router } from '@angular/router';
3 import { ContactService } from '../services/contact.service';
4 import { Contact } from '../Shared/contact';
5
6 @Component({
7   selector: 'app-contact-detail',
8   templateUrl: './contact-detail.component.html',
9   styleUrls: ['./contact-detail.component.css']
10 })
11 export class ContactDetailComponent implements OnInit {
12   idContact: any;
13   contact: Contact;
14   constructor(private contactService: ContactService,
15               private route: ActivatedRoute,
16               private router: Router) { }
17   ngOnInit(): void {
18     /** snapshot method */
19     // this.idContact=this.route.snapshot.params.id;
20     /** asynchronous method */
21     this.route.paramMap.subscribe(res => {
22       this.idContact = res.get('id');
23     })
24     this.contact = this.contactService.getContactById(this.idContact);
25     console.log(this.contact);
26   }
27   onContacts() {
28     this.router.navigate(['/contacts']);
29   }
30 }
```

#05

```
# Création du contactDetailComponent
$ ng generate component contactDetail
```

```
< contact-detail.component.html X
src > app > contact-detail > contact-detail.component.html > div.card.border-primary.mb-3
1 <div class="card border-primary mb-3">
2   <div class="card-header bg-primary text-white"> Contact Detail Component</div>
3   <div class="card-body">
4     <div *ngIf="contact ; else noContacts">
5       <h3>Contact detail</h3>
6       <div class="card" style="width: 25em;">
7         
8         <div class="card-body">
9           <h5 class="card-title">{{contact.name}}</h5>
10          <p class="card-text"><strong>Projects :</strong> {{contact.projects}}.</p>
11        </div>
12          <ul class="list-group list-group-flush">
13            <li class="list-group-item"><strong>Email :</strong>{{contact.email}}</li>
14            <li class="list-group-item"><strong>Webiste :</strong>{{contact.website}}</li>
15          </ul>
16        </div>
17      </div>
18      <ng-template #noContacts>
19        <p>Contact not found</p>
20      </ng-template>
21    </div>
22  </div>
23  <button class="btn btn-primary" (click)="onContacts()">
24    Go to Contacts
25  </button>
```

Solution

#04

Entant que utilisateur je veux
voir la liste des contacts

#05

Entant que utilisateur je veux
voir les détails d'un contact

#06

Entant que utilisateur je veux
supprimer un contact

5 Mettez à jour **app-routing.module.ts**

```
src > app > TS app-routing.module.ts > ...
1  import { NgModule } from '@angular/core';
2  import { Routes, RouterModule } from '@angular/router';
3  import { AboutComponent } from '../about/about.component';
4  import { ContactDetailComponent } from '../contact-detail/contact-detail.component';
5  import { ContactsComponent } from '../contacts/contacts.component';
6  import { HomeComponent } from '../home/home.component';
7  import { NotFoundComponent } from '../notfound/notfound.component';
8  import { SigninComponent } from '../signin/signin.component';
9
10 const routes: Routes = [
11   { path: '', component: HomeComponent },
12   { path: 'contacts', component: ContactsComponent },
13   { path: 'contacts/:id', component: ContactDetailComponent },
14   { path: 'about', component: AboutComponent },
15   { path: 'signin', component: SigninComponent },
16   { path: '**', component: NotFoundComponent }
17 ];
18
19 @NgModule({
20   imports: [RouterModule.forRoot(routes)],
21   exports: [RouterModule]
22 })
23 export class AppRoutingModule {
24 }
```

Résultat : Contacts & ContactDetail Component

#04

Entant que utilisateur je veux
voir la liste des contacts

#05

Entant que utilisateur je veux
voir les détails d'un contact

#06

Entant que utilisateur je veux
supprimer un contact

Contacts Component		
Contact list		
Name		
Doug Cutting	Detail	Delete
Bill Gates	Detail	Delete
Mark Zuckerberg	Detail	Delete

Contact Detail Component

Contact detail



Doug Cutting

Projects : Nuch,Hadoop.

Email : doug.cutting@cloudera.com

Webiste : http://hadoop.apache.org/

[Go to Contacts](#)

02

Guards



Guard : définition

- Un **Guard** dans Angular est une fonctionnalité qui permet de contrôler l'accès à des routes spécifiques dans une application. Souvent les guards sont utilisés pour exécuter certaines vérifications ou actions avant de permettre l'accès à une route, par exemple pour vérifier si l'utilisateur est authentifié ou a les droits d'accès appropriés (privilèges).
- Il existe plusieurs type de **Guard** :
 - **CanActivate** : vérifie si un utilisateur peut visiter une route.
 - **CanDeactivate** : vérifie si un utilisateur peut quitter une route.
 - **CanActivateChild** : vérifie si un utilisateur peut visiter les routes enfants.
 - **CanLoad** : vérifie si un utilisateur peut aller sur une route d'un module défini avec un lazy loading

Guard : AuthGuard

Exemple :

- On veut que l'accès à la route **/contacts** soit seulement autorisé aux utilisateurs authentifiés
- On va créer un **Guard** Angular de type **CanActivate** et l'associer à la route **/contacts** dans **app-routing.module.ts**

Il faut mettre la condition qui active le lien

```
# AuthGuard
$ ng generate guard guards/auth
? Which type of guard would you like to create? (Press <space> to select, <a> to toggle all, <i> to invert selection, and <enter> to proceed)
>(*) CanActivate
  ( ) CanActivateChild
  ( ) CanDeactivate
  ( ) CanMatch
```

```
TS auth.guard.ts U x
src > app > guards > TS auth.guard.ts > ...
1 import { CanActivateFn } from '@angular/router';
2
3 export const authGuard: CanActivateFn = (route, state) => {
4   return true;
5 };
```

```
{path: 'contacts', component: ContactsComponent, canActivate: [AuthGuard]},
```


AuthGuard : Gestion d'accès

#07

Entant que utilisateur je veux
me connecter

#03 BNF

Gérer les droits d'accès (seul
l'utilisateur connecté peut consulter
home, about et contacts)

first-project Home About Contacts Signin

SignIn

SingOut

AuthService : Etape 1

- Créer un service d'authentification
 - Le service d'authentification est utilisé pour se connecter et se déconnecter d'une application Angular,
 - Il notifie les autres composants lorsque l'utilisateur se connecte et se déconnecte, et permet d'accéder à l'utilisateur actuellement connecté
 - Dans notre exemple, nous allons nous limiter aux 3 fonctions :
 - `signIn()` et `signOut()` : affectent respectivement `true` et `false` à l'attribut `isAuth`
 - `isAuthenticated()` : retourne la valeur de `isAuth`



The image shows a terminal window and a code editor. The terminal window at the top has a title bar with three colored circles (red, yellow, green) and contains the following text:

```
# Création de AuthService  
$ ng generate service services/auth
```

Below the terminal is a code editor window titled "TS auth.service.ts U X". The editor shows the following TypeScript code:

```
1 import { Injectable } from '@angular/core';  
2  
3 @Injectable({  
4   providedIn: 'root'  
5 })  
6 export class AuthService {  
7   private isAuth:boolean=false;  
8  
9   constructor() { }  
10  
11   signIn(){  
12     this.isAuth=true;  
13   }  
14  
15   signOut(){  
16     this.isAuth=false;  
17   }  
18  
19   isAuthenticated():boolean{  
20     return this.isAuth  
21   }  
22 }
```

A yellow sticky note with the text "#07" is placed over the right side of the code editor.

SignInComponent : Etape2

- Mettre à jour SignInComponent

```
src > app > signin > < signin.component.html > ...
1 <div class="form-group">
2   <button class="btn btn-success" (click)="onSignIn()" [disabled]="isAuth"> SignIn</button>
3 </div>
4 <div class="form-group">
5   <button class="btn btn-danger" (click)="onSignOut()" [disabled]="!isAuth"> SingOut</button>
6 </div>
```

first-project Home About Contacts Signin

SignIn

SingOut

©MAD

```
TS signin.component.ts U X #07
src > app > signin > TS signin.component.ts > SignInComponent
1 import { Component, OnInit } from '@angular/core';
2 import { AuthService } from '../services/auth.service';
3 @Component({
4   selector: 'app-signin',
5   templateUrl: './signin.component.html',
6   styleUrls: ['./signin.component.css']
7 })
8 export class SignInComponent implements OnInit {
9   isAuth:boolean=false;
10  constructor(private authService: AuthService){}
11  ngOnInit(): void {
12    this.isAuth=this.authService.isAuthenticated()
13  }
14  onSignIn():void{
15    this.authService.signIn();
16    this.isAuth= this.authService.isAuthenticated();
17  }
18  onSignOut():void{
19    this.authService.signOut();
20    this.isAuth= this.authService.isAuthenticated();
21  }
22 }
```

AuthGuard : Etape3

- Créer le Guard **AuthGuard**

TS auth.guard.ts U X

src > app > guards > TS auth.guard.ts > ...

```
1 import { CanActivateFn, Router } from '@angular/router';
2 import { AuthService } from '../services/auth.service';
3 import { inject } from '@angular/core';
4
5 export const authGuard: CanActivateFn = (route, state) => {
6   const auth = inject(AuthService);
7   const router = inject(Router);
8
9   if(!auth.isAuthenticated()) {
10     router.navigateByUrl('/signin')
11     return false
12   }
13   return true;
14 };
```

```
# AuthGuard
$ ng generate guard guards/auth
? Which type of guard would you like to create? (Press <space> to select, <a> to toggle all, <i> to invert selection, and <enter> to proceed)
>(*) CanActivate
  () CanActivateChild
  () CanDeactivate
  () CanMatch
```

#03 BNF

Si utilisateur connecté,
retourner **True**
Sinon, rediriger utilisateur
vers la page de connexion

AppRoutingModule : Etape4

- Activer les **Guards** dans le module de **Routage**

```
TS app-routing.module.ts M X
1 import { NgModule } from '@angular/core';
2 import { RouterModule, Routes } from '@angular/router';
3 import { AboutComponent } from '../about/about.component';
4 import { ContactsComponent } from '../contacts/contacts.component';
5 import { HomeComponent } from '../home/home.component';
6 import { SigninComponent } from '../signin/signin.component';
7 import { NotFoundComponent } from '../notfound/notfound.component';
8 import { ContactDetailComponent } from '../contact-detail/contact-detail.component';
9 import { authGuard } from '../guards/auth.guard';
10
11 const routes: Routes = [{path: '', canActivate: [authGuard], component: HomeComponent},
12   {path: 'about', canActivate: [authGuard], component: AboutComponent},
13   {path: 'contacts', canActivate: [authGuard], component: ContactsComponent},
14   {path: 'contacts/:id', canActivate: [authGuard], component: ContactDetailComponent},
15   {path: 'signin', component: SigninComponent},
16   {path: '**', component: NotFoundComponent}];
17
18 @NgModule({
19   imports: [RouterModule.forRoot(routes)],
20   exports: [RouterModule]
21 })
22 export class AppModule { }
```

#03 BFN

03

Atelier 3

